

תכנון בתור סיפוק אילוצים

עד עכשיו הסתכלנו על בעיות תכנון בתור בעיות של חיפוש בעץ. אבל אפשר גם להסתכל עליהן כעל בעיות CSP¹ - כאשר הקודקוד שמוצאים (שמספק את האילוצים) הוא בעצם התוכנית.

בעיות חיפוש מסלול אפשר לחפש משני כיוונים:

1. מתחילים מהמצב ההתחלתי, ומפתחים עד שמגיעים עד לgoal state.

בעיה: אנחנו מפתחים המון כיוונים שלא יקרבו אותנו לפתרון.

זה מה שלמדנו עד עכשיו.

2. regression state - מתחילים מהמטרה, ופתחים אחורה כדי למצוא את שרשרת הפעולות שתוביל אותנו מהמצב ההתחלתי.

יש גישות שמשלבות - לוקחים צעד של forward וצעד של backward ומפתחים עד שהן נפגשים.

קונסטראינטיות, initial state יוצר אילוץ: מסלול שלא עובד דרכו לא עוזר לנו. goal state גם הוא אילוץ כזה.

הרעיון: נניח שקיימת תוכנית בגודל k צעדים שמגיעה מהמצב ההתחלתי $State_0$ למצב המטרה $State_k$.

• הפתרון הוא גרף CSP שבו:

- $k + 1$ שכבות מצב - כל שכבה מורכבת מכל state variable.
- בין כל שכבת מצב, יש משתנה פעולה - פעולה אפשרית.
- בין כל שכבת מצב למשתנה פעולה יש שכבה של קודקודי אילוץ - משני הכיוונים:
- * בין מצב לפעולה - האילוצים הם הpre-conditions של הפעולה.
- * בין פעולה למצב - האילוצים הם האפקטים של הפעולה.
- יש גם קודקודי אילוץ ישירים בין כל משתנה מצב למשתנה המצב בשלב הבא - שדואג שהוא לא ישתנה אם הפעולה לא נוגעת בו.

פורמלית, לכל $State_i$ (כאשר $0 \leq i \leq k$) יהיו לנו:

- N משתני מצב. אלו לא המצבים, אלא הpropositions שמרכיבים את המצב.
- לכל $1 \leq i$ יהיה משתנה פעולה $Action_{i-1}$, שהערך שלו הוא אחת מ- $|A|$ הפעולות האפשריות.

האילוצים: $State_0$ הוא המצב ההתחלתי

• $State_k$ הוא מצב מטרה (יכולים להיות כמה מצבים - האילוצים מגדירים)

• עבור פעולה $Action_{i-1}$:

- אילוצים לפי preconditions בין הקודקוד לקודקודי המשתנים ב- $State_{i-1}$.

- אילוצים לפי effects בין הקודקוד לקודקודי המשתנים ב- $State_i$.

¹סיפוק אילוצים - Constraint Satisfaction Problems

• frame constraints - אם פעולה לא נוגעת במשתנה, אז הוא נקבע לפי הערך הקודם שלו.

נשים לב:

- אם לא יודעים את k , צריך להתחיל מ $k = 1$ ואם לא מוצאים פתרון - לעלות ל $k + 1$. אבל נשים לב שזה נותן לנו אורך מינימלי ולא עלות מינימלית.
- אין כאן כיוון חיפוש כי אנחנו לא מחפשים מסלול.
- אפשר לפתור בצורה מהירה באמצעות iterative repair - אבל זה local search אז אם לא מצאנו פתרון זה לא מבטיח שבאמת אין פתרון.

אפשרות לשיפור:

1. מספר פעולות במקביל. כל סוג של פעולה הופך למשתנה נפרד, שמקבל ערכים בוליאניים. יהיו אילוצים בין הפעולות, שיוודאו שלא מבצעים שתי פעולות באותה שכבה.
2. ערכים נומריים. למשל - כמות דלק במיכל, שאם אין מספיק אז אסור להתחיל בנסיעה.
3. כל מיני תנאים שחייבים להתקיים במהלך המסלול.
4. לתאר את זה בתור בעיית SAT.

Branch and Bound

Branch and Bound היא משפחה של אלגוריתמים שבה יש:

- צעדי branch, שבהם מפתחים צמתים שרוצים לבדוק.
- צעדי bound, שבהם חוסמים צמתים לפי פרמטרים מסויימים - וככה קוטמים ענפים שאנחנו לא רוצים לבדוק.

הערה: A^* הוא למעשה מקרה פרטי של Branch and Bound.

תזכורת: backtracking בנוי מעל DFS, כאשר בוחרים בצורה חכמה למה להיכנס.

נרצה להפוך את האלגוריתם הרקורסיבי לאלגוריתם איטרטיבי באמצעות מחסנית. משתמשים בשתי מחסניות:

- $OPEN$ - המחסנית הראשית. דוחפים אליה את השורש, וכל עוד היא לא ריקה לוקחים את האיבר בראשה, בודקים אם הוא complete (מקיים את תנאי האילוץ), אם כן אז הוא התשובה - אבל אם לא מפתחים אותו ודוחפים את השכנים חזרה ל- $OPEN$.

- R - מחסנית שמשמשת כדי להפוך את הקודקודים שמכניסים למחסנית בכל פיתוח. היוריסטיקה נותנת קודם את הקודקודים שאנחנו רוצים לבדוק קודם - אבל אז הם יהיו עמוק יותר במחסנית ונגיע אליהם יותר מאוחר. לכן משתמשים ב- R בשביל לעשות היפוך.

אפשר אמנם לעשות את זה בצורה יותר יעילה אם, למשל, הופכים את היוריסטיקה. אבל זה דורש להתערב ביוריסטיקה. בהמשך ניפתר מזה.

למה בעצם צריך להתחייב קודם למשתנה ואז לבחור את הערכים שלו? כי אם נפתח תמיד את כל האפשרויות לכל המשתנים, זה יגדיל לנו יותר מדי את ה- $OPEN$. כדי להתמודד עם הגודל, נרצה להחזיק תיעוד "הדברים שעוד לא חישבתי" ולדחות את החישוב לזמן שנצטרך. זה חלק מרעיון כללי שנקרא lazy computation. אלגוריתם החיפוש הנאיבי (בסדר שרירותי) לא מפריד בין משתנים והשמות. עבור כל המשתנים הפתוחים, עבור כל הערכים האפשריים, הוא יוצר קודקוד השמה ודוחף ל- $OPEN$. זה מאוד לא יעיל - אבל זו המסגרת.

תזכורת: ה-Bound $B()$ מקיים:

- $B(n) = \infty$ אם $\text{Consistent?}(n) = \perp$
- $B(n) < \infty$ אם $\text{Consistent?}(n) = \top$

ב-CSP, $B(n) = 1$ אם $\text{Complete?}(n) = \perp$ ו- $B(n) = 0$ אם $\text{Complete?}(n) = \top$.

נרצה להרחיב לאילוצים קשים ורכים:

- $B(n) = \infty$ אם אילוץ קשה מופר.
 - $B(n) = \#$ אם רק אילוצים רכים מופרים, כאשר $\#$ הוא מספר האילוצים המופרים.
- אפשר להכליל עוד יותר ולתת משקלים להפרה. לכל $c \in C$ משקל ההפרה הוא $w_c(n)$, ואז

$$B(n) = \sum_{c \in C} w_c(n)$$

עכשיו, אפשר לשמור תמיד את ה- $B(n)$ הכי טוב - ולא לפתח דברים קודקודים פחות טובים.

- בעצם: פתרונות מלאים נותנים לנו upper bound
- פתרונות חלקיים נותנים לנו lower bound

בהתחלה best ה- ∞ הוא, ואז ה- $OPEN$ גדל וגדל - עד שמוצאים פתרון שלם ראשון שמאפשר לנו לעשות pruning לפתרונות - חלקיים או שלמים - שנותנים דברים פחות טובים.

יש לנו כאן בעיה מובנית - הגודל של $OPEN$. נרצה דרך להימנע מפיצוץ ה- $OPEN$ בכל האפשרויות לפני שאפשר להתחיל לפלטר.

הטכניקה הכללית נקראת Deferred Computation. במקום רשימה שמחזיקה את הכל בזיכרון, משתמשים באובייקט שנותן בכל פעם את האובייקט הבא. בשביל לבצע את זה, נשתנה את ה- $search\ node$: במקום $n = \langle A, V, C \rangle$, נגדיר $n = \langle A, V, C, U(v) \rangle$ כאשר $U(v)$ מייצג את $Unassigned(n, v)$ - סט הערכים של v שעדיין לא בדקנו.

- הרעיון: לכל $v \in V$, מתחילים את $U(v)$ בכל הערכים ש- v יכול לקבל.
- בכל פעם שמגיעים לפתרון n שאינו שלם ומפתחים עליו ערכים של v - מעדכנים את $U(v)$.
- כשבוחרים משתנה, בוחרים או מתוך משתנים חופשיים (שאינן להם השמות) או מתוך משתנים שכבר יש להם השמות - אבל יש להם ערכים שעוד לא ניסו.
- לא מכניסים את כל הילדים - רק את k הערכים הטובים ביותר. כל השאר נשארים ב- $U(v)$.
- יכול להיות שניתקל שוב בצומת שכבר בדקנו - פשוט כי עוד לא ניסו את הכל. אמנם מבחינה מתמטית זה לא אותו אובייקט, אבל מבחינה תכנותית זה אותו אובייקט (הוא לא immutable)

תכנון כחיפוש קודקוד במרחב תוכניות

כל צומת במרחב הזה תייצג תוכנית, ונחפש צומת שבה יש תוכנית מלאה (נשים כרגע אופטימליות בצד) אפשר לייצג את הבעיה כגרף של מצבים, אבל אפשר גם לייצג מרחב של תוכניות אפשריות:

- מתחילים מקודקוד שבו תוכנית ריקה.
- בכל פעם נסתכל על צומת, נבין מה לא בסדר איתה, ונייצר צמתים שמתקנים אותה:
 - אפשר לתקן באמצעות ייצור פעולה.
 - אפשר לסדר מחדש את הפעולות.
 - (אפשר גם להוריד פעולה - אבל באופן עקרוני לא נעשה את זה)
- עוברים בלולאה, שכל פעם מוצאת flaws ומייצרת דרכים לתקן.

נגדיר: $\pi := \langle A, O, L \rangle$ כאשר:

A	מולטי-סט ² של פעולות ב Ω , סט ground actions האפשריות.
O	סט של ordering links: $(a_1, a_2) \in O$ אומר $a_1 \in A$ מתרחש לפני $a_2 \in A$.
L	סט של causal links. כלומר $[a_1 \rightarrow^c a_2] \in L$ אומר $a_1 \in A$ ממלא את c שהוא precondition של $a_2 \in A$.

צומת חיפוש במרחב הזה מכיל:

- תוכנית π - שיכולה להיות ואלידית או לא-ואלידית.
- אג'נדה A - רשימת של פגמים בתוכנית. $A = \emptyset$ אומר שהתוכנית ואלידית.

יש שני סוגי פגמים:

סוג 1: $\langle c, a_j \rangle$ open precondition. כלומר:

- c הוא תנאי מקדים של $a_j \in \pi$
- אין causal link $l \in L$ שמספק את c

סוג 2: causal link לא בטוח $l = [a_i \rightarrow^c a_j]$. כלומר:

- $\exists a_k \in O$ שנמצא בין a_i ל a_j
- $\neg c \in \text{preconds}(a_k \in A)$

התוכנית ההתחלתית היא $\pi_0 := \langle \{\text{start}, \text{end}\}, \emptyset, \emptyset \rangle$

start פעולה שאין לה preconditions והeffects שלה הם המצב ההתחלתי.

end פעולה שאין לה effects, והpreconditions שלה הם מצב מטר.

²כלומר קבוצה אם אפשרות לכפילויות.

הצומת ההתחלתי הוא $(\pi_0, \mathbb{A}_0)$ כאשר:

$$\mathbb{A}_0 := \{ \langle \text{pre}_i(\text{end}), \text{end} \rangle \mid \text{pre}_i \in \text{preconds}(\text{end}) \}$$

האלגוריתם: באיטרציה/רקורסיה:

- בוחרים פגם בתוכנית.
- אם הפגם הוא מסוג open precondition:
 - עושים 3ND-CHOOSE לפעולה.
 - אם אין פעולה כזאת - הנתיב הזה מוביל לכישלון.
 - אם היא לא בא A - מוסיפים אותה לא.
 - מוסיפים את causal link והordering constraint שצריך בשביל לדאוג שהפעולה תתקל את הפגם.
 - מוסיפים את הפעולה, את ordering constraint, ואת causal link לתוכנית.
- אם הפגם הוא מסוג unsafe causal link:
 - עושים ND-CHOOSE לordering constraint.
 - אם אין כזה - הנתיב מוביל לכישלון.
 - מוסיפים את ordering constraint לתוכנית.
- נשים לב: שיש כאן דמיון מפתיע לתהליך חיפוש הקודקוד Branch and Bound. אמנם חייבים לטפל בכל הפגמים שבתוכנית - אבל בכל פעם בוחרים פגם אחד ומטפלים בו.
- התוכנית היא partially ordered - זה מבנה קומפקטי שמייצג את כל הסדרים האפשריים, ולכן הביצוע מאפשר למקבל.
- חסרונות: – מרחב החיפוש אינסופי, כי תמיד אפשר לייצר עוד תוכניות. – אין יוריסטיקות טובות.
- אפשר לשפר עם טכניקות כמו יוריסטיקות state-space ועם landmarks - אבל זה לא מספיק טוב להתחרות עם אלגוריתמים חדשים שרצים ב-state-space.